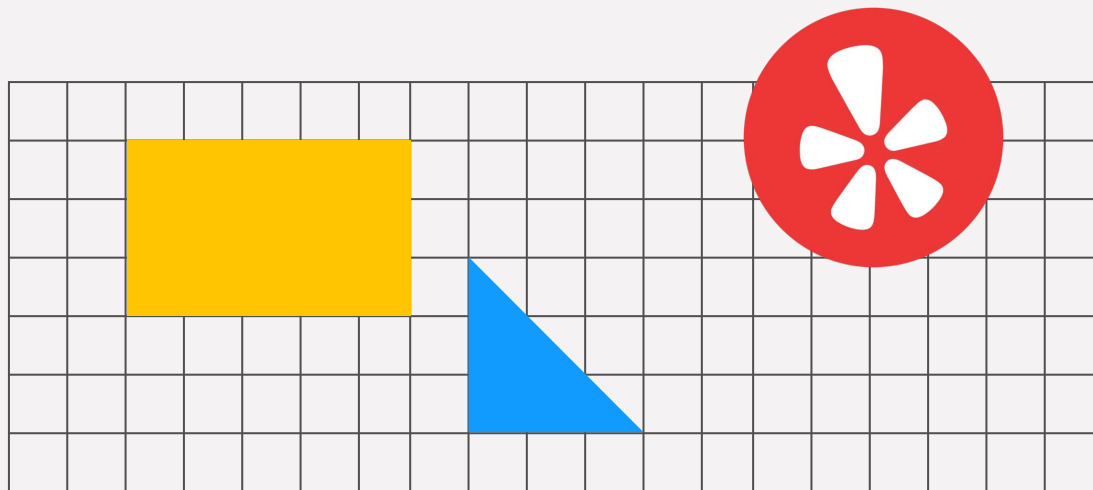


AgentSociety Challenge: Yelp Recommendation Agent

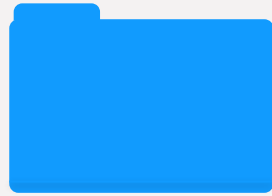
CS245

Eliot Yoon, Ryan Wang

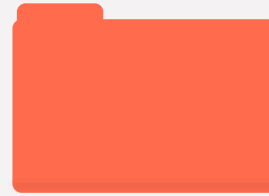




Introduction



Methodology



Results



Future Work

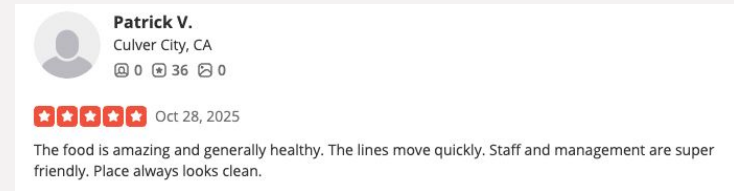
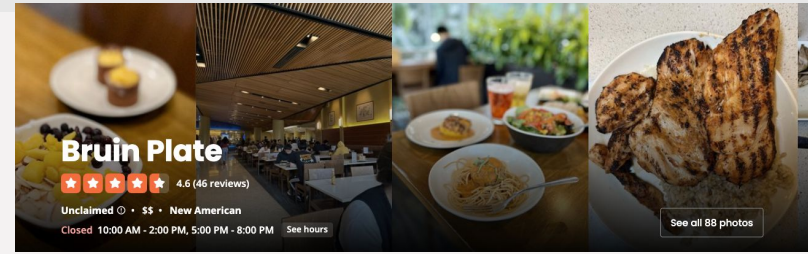
Introduction

- **Recommendation track:** building LLM agents that can provide personalized recommendations, based on user preferences
- **Yelp** is an online crowdsourced platform that allows people to discover, rate, and review local businesses

Dataset

The Yelp Dataset is split into 3 Tables of data

- **User Data:** Metadata about the yelp user
 - each item includes name, review count, account creation date, etc.
- **Item Data:** Data about businesses
 - each item includes business **name**, **average stars**, **hours**, etc.
- **Review Data:** User reviews of businesses
 - Each review item includes a star rating, text description of the user's experience of the business, etc.
 - Each review item is associated with a user and a business



Goal

- Given a user and a candidate item list, predict the user's preference order of the candidate items

Methodology/System design

To motivate the design of our model, we investigated the following techniques:

- Feature/prompt engineering
- Reasoning
- Memory

Feature Engineering

Key Insight: Uninformative features increase noise of the input (the LLM prompt)

- Using only key features increases signal to noise of input, improving inference

Baseline

- does not utilize User data
- all the features of all the user's reviews are used
- For the item table, Item_id, name, stars, review_count, and all attributes are used
- in user reviews, features such as review_id have no semantic value and only divert attention from important features
- In items, important features like business categories (item_categories) are omitted

Feature Engineering (cont.)

Key Insight: reviews linked to an item are an unutilized source of information

- We utilize this information by using an LLM to summarize these reviews and utilizing this text as an item feature
 - Encapsulates information about business not captured in provided quantitative features

Feature Engineering (cont.)

Key Insight: Filter and Stringify data in a maximally informative way

- LLMs are most adept at reasoning with natural language
 - Rather than prompting the model with the raw JSON string, output should be presented in natural language

Reasoning

Key Insight: Each task can be broken up into multiple reasoning subtasks

- Allows for each specific subtask to be performed in isolation without being influenced by the other subtasks

User Reasoning

- Takes each user's data and historical reviews and generates a summary of their tastes, preferences, and other pertinent information
- Reduces input token size from just passing in all raw data and isolates important information

Reasoning (cont.)

Item Reasoning

- Similar to user reasoning, summarizes the attributes and reviews of each candidate
- Allows for LLM to have greater context of general sentiment about each candidate

Task Reasoning

- Final reasoning module that outputs the ranking
- Utilizes the outputs of previous user and item reasoning subtasks as features in determining the final ranking
- Specificity of instructions given to LLM in prompt is very important
 - Significant performance improvement from telling LLM to weigh candidate ratings more than user preferences

Memory

Key Insight: Memory can be used to avoid repetitive LLM calls and reduce costs

User and Item Memory

- Outputs of user and item reasoning modules stored in memory
- Uses langchain_chroma to maintain persistent memory
- Candidate items are often repeated, so item reasoning module can search memory for existing summary and avoid another LLM call if it exists
- Also greatly reduces cost and execution time during experimentation of other ablations

Results

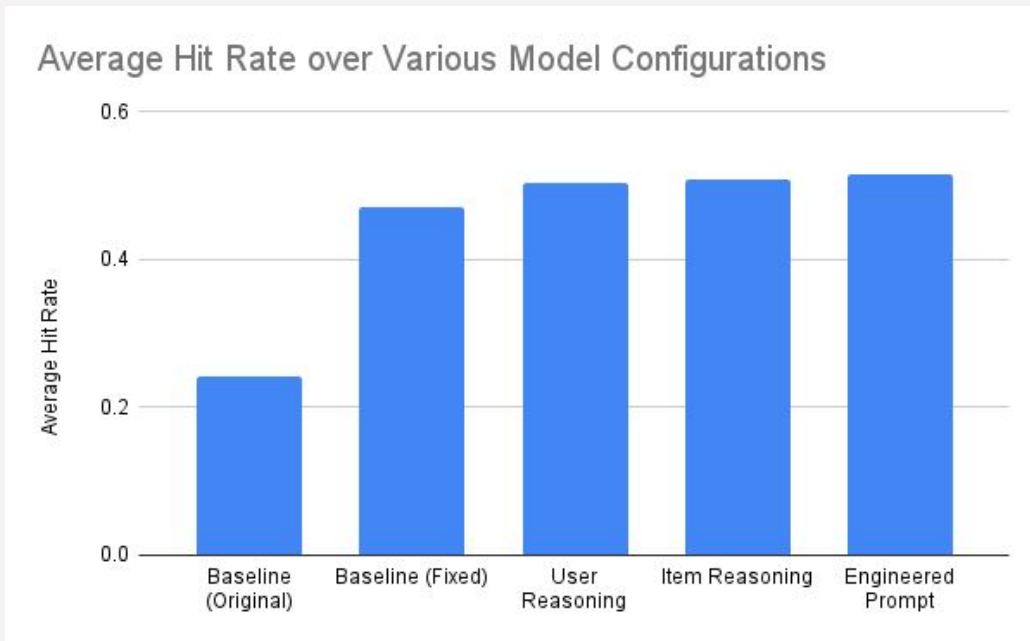
Key Insight: Our best performing model was a culmination of all of the previously mentioned techniques

Evaluation Metrics (HR@K)

-

	Baseline (Original)	Baseline (Fixed)	Our Model	% Increase
Top-1	0.0755	0.2604	0.3021	16.01
Top-3	0.2240	0.5078	0.5677	11.80
Top-5	0.4297	0.6458	0.7109	10.08
Avg	0.2431	0.4714	0.5269	11.77

Results (cont.)



Analysis

Performance:

- Our results show a noticeable improvement in prediction accuracy
- The biggest improvement came from combining user + item reasoning and engineering the task prompt with greater specificity

Efficiency:

- On first evaluation, model must summarize candidates for each task
 - Up to 20 additional API calls per task, slow and very high costs
- Shows importance of using memory to store reusable data for future evaluation

Future Work

Self-Improvement

- Use already completed tasks to learn how to better solve similar problems

Further Prompt Engineering

- Further specify prompt instructions so that LLM has a more structured plan of action

Efficiency

- Find ways to combine/shorten API calls to reduce cost and improve efficiency



Thank you!